II PLL 180 MHz

Contreras R Orlando[1,2*] and Lara H Kevin[1,2*]

[3]Department of Electronic Systems, UAA, Av. Universidad 940, Aguascalientes, 20131, Aguascalientes, México.

*Corresponding author(s). E-mail(s): {al348390,al464376}@edu.uaa.mx;

Abstract

The objetive of this lab practice is change the frequency of the main clock to the MAX, handling preescalers and wait states, this will alow us to do task at a higher frequency, even do multiple task by concurrency, we could prove this by reading the signal by an oscilloscope.

Keywords: C, ARM, Latency, Preescalers, Frequency

1 Introduction

In this practice we will configure the main clock to 180 MHz (Max Speed), to do this we would need to configure the RCC and PLL as requires. Using the PLL parameters as PLLN,PLLM and PLLP and using the Reference Manual formula we can obtain clock frecuency of 180MHzand send it to an LED output and see it in the osciloscope. The osciloscope will return to us the frecuency and with some mathematical calculations obtain the original clock frecuency.

2 Objetives

- Correct use of RCC, preescalers and Clocks.
- Correct use of GPIO.
- Perform a general formula with the condition parameters of PLLN, PLLM, PLLP to calculate the frequency desired.
- Develop a precise delay due to the clock.

3 Method

3.1 Pseudocode

Here is the pseudo code that represents the algorithm, also it is included the configuration as masks or

Algorithm 1 System Configuration

Input: None

Output: System clock and GPIO configuration

```
setClkPLL() // Configure PLL clock
confRCC() // Enable peripheral clocks
confGPIO() // Setup GPIO pins
```

Algorithm 2 RCC Configuration

Input: None

Output: Enable GPIOA clock

```
confRCC() // Enable peripheral clocks
```

$RCC.AHB1ENR \leftarrow RCC.AHB1ENR \vee (1 \ll 0)$ // Enable GPIOA

Algorithm 3 GPIO Configuration

Input: None

Output: Configure PA5 and PA6 as output

```
confGPIO() // Setup GPIO pins
```

$GPIOA.MODER \leftarrow GPIOA.MODER \vee (5 \ll 10)$ // Set PA5-6 as output

$GPIOA.OSPEEDR \leftarrow GPIOA.OSPEEDR \vee (3 \ll 10)$ // High speed

Algorithm 4 PLL Configuration

Input: None

Output: Configure PLL for high-speed clock

```
setClkPLL() // Configure PLL clock
```

$RCC.CR \leftarrow RCC.CR \vee (1 \ll 16)$ // Enable HSE

while $\neg(RCC.CR \wedge HSERDY)$ **do**

// Wait HSE ready

$FLASH.ACR \leftarrow FLASH.ACR \vee (1 \ll 8)$ // Prefetch enable

$FLASH.ACR \leftarrow FLASH.ACR \vee (5 \ll 0)$ // 2 wait states

$RCC.CFGR \leftarrow RCC.CFGR \vee (5 \ll 10)$ // APB1 prescaler

$RCC.CFGR \leftarrow RCC.CFGR \vee (4 \ll 13)$ // APB2 prescaler

$RCC.PLLCFGR \leftarrow RCC.PLLCFGR \vee (1 \ll PLLSRC)$ // PLL source

$N \leftarrow 360$, $M \leftarrow 8$, $P \leftarrow 0$ // PLL params

$RCC.PLLCFGR \leftarrow RCC.PLLCFGR \vee (N \ll PLLN)$ $RCC.PLLCFGR \leftarrow RCC.PLLCFGR \vee (M \ll PLLM)$ $RCC.PLLCFGR \leftarrow RCC.PLLCFGR \vee (P \ll PLLP)$ $RCC.CR \leftarrow RCC.CR \vee (1 \ll 24)$ // Enable PLL

$RCC.CFGR \leftarrow RCC.CFGR \vee (2 \ll 0)$ // Switch to PLL

```
SysClkUpdate() // Update clock var
```

3.2 Register Address

- **FLASH**: Register used to enable/disable acceleration features due to CPU Frequency.
- **RCC**: Base address of clock control register.
- **RCC_AHB1ENR**:Register used to enable port clocks.
- **Registers MODER, PUPDR**: Used to configurate GPIOs.
- **Registers IDR y ODR**: Input and ouput data registers .

3.3 System Initialization

The main code begins by enabling peripheral's clock of **PORTA** by the `RCC_AHB1ENR` register.

```
RCC->AHB1ENR |= (1 << 0);
```

3.4 Configuration of GPIO

- The pins A5, A6 are setted as output mode with High Speed

```
GPIOA->MODER |= (5 << 10);
GPIOA->OSPEEDR |= (3 << 10);
```

3.5 PLL Config

We need to enable the High Speed Oscillator (HSE) and wait until the HSE is stable (HSE_RDY)

```
RCC->CR |=(1 << 16);
while(!(RCC->CR & RCC_CR_HSERDY));
```

Then we need to enable prefetch on FLASH Register. The prefetch, as the name says, prefetch each block of instructions instead of waiting until the block end. That's mean that we don't have to wait until the first 4 instructions to start fetching the next block of 4(see Figure 1). Despite of that we need to implement a minimum of wait states acording to the speed of the embedded flash memory (25-30 ns) with a cycle of 180 MHz would be like 5.55 ns requiring as minimum 5 wait states, in the reference manual we got a table that indicates the suggested wait states for each frequency.

```
FLASH->ACR |= (1<<8);
FLASH->ACR |= (5<<0);
```

After that we need to configure the RCC Preescalers, we have to consider the conditions of APBs

$$APB1 \leq 45MHz$$
$$APB2 \leq 90MHz$$

To get the value of 180 MHz we need to preescale the first APB by 4, while the second by 2.

```
RCC->CFGR |= (5<<10);
RCC->CFGR |= (4<<13);
```

Figure 4. Sequential 32-bit instruction execution

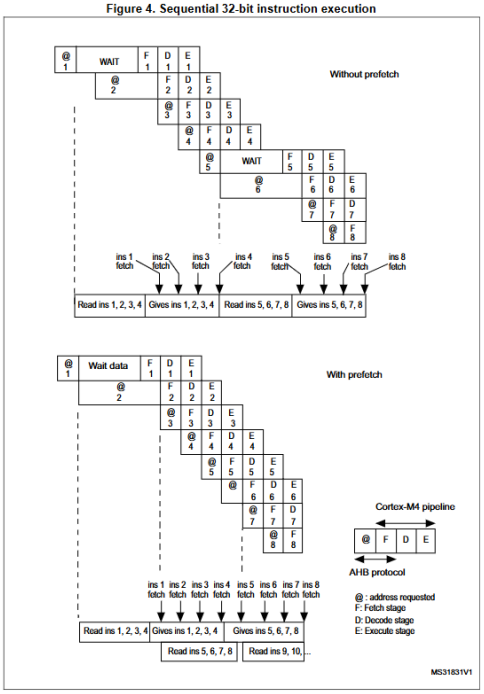

Fig. 1: Prefetch Cycle

Then we configure the PLL's Registers, we set the source of the PLL as the HSE instead of the HSI

```
RCC->PLLCFGR |= (1 << RCC_PLLCFGR_PLLSRC_Pos);
```

We have a formula [**eq.** 1] for calculate the Clock Output due to PLLM,PLLN and PLLP parameters.

$$f_{PLL\_Out} = \frac{HSE}{PLLM \times PLLP} \times PLLN \tag{1}$$

Considering that the clock of HSE is of 8MHZ and we desire 180 MHz we need a value that

$$8x = 180 \rightarrow x = \frac{180}{8} = \frac{90}{4} = \frac{360}{16}$$

being $x = \frac{N}{MP}$, with the following parameters.

$$N = [100, 432], M = [2, 63], P \in \{2, 4, 6, 8\}$$

Here we got a lot of options, it could be

$$\frac{N[180]}{M[2]P[4]} = \frac{N[360]}{M[4]P[4]} = \frac{N[360]}{M[2]P[8]}$$

```
RCC->PLLCFGR |= (N << RCC_PLLCFGR_PLLN_Pos) |
                (M << RCC_PLLCFGR_PLLM_Pos) |
                (P << RCC_PLLCFGR_PLLP_Pos) ;
```

Finally we turn on the PLL in the RCC Register and select the PLLP as the main clock and update the clcoks

```
  RCC->CR   |= (1 << 24);
  RCC->CFGR |= (2 << 0);
  SystemCoreClockUpdate();
```

4 Results and Discussion

4.1 Code

Here's the code that represents the previous pseudocode

Code: `conf.c`

```
#include <stm32f446xx.h>
#include "conf.h"

void confRCC(void){
  RCC->AHB1ENR |= (1 << 0); // A  E
}
void confGPIO(void){
  GPIOA->MODER |= (5 << 10);
  GPIOA->OSPEEDR |= (3 << 10);
}

void setClkPLL(void)
{
  RCC->CR |=(1 << 16);
  while(!(RCC->CR & RCC_CR_HSERDY));
  FLASH->ACR |= (1<<8) ;
  FLASH->ACR |= (5<<0);
  RCC->CFGR |= (5<<10);
  RCC->CFGR |= (4<<13);
  RCC->PLLCFGR |=(1 << RCC_PLLCFGR_PLLSRC_Pos);
  uint8_t N, M, P;
  N=360; M=8; P=0; /*P[0]=2 P= {2,4,6,8}*/

  RCC->PLLCFGR |= (N << RCC_PLLCFGR_PLLN_Pos) |
          (M << RCC_PLLCFGR_PLLM_Pos) |
          (P << RCC_PLLCFGR_PLLP_Pos) ;
  RCC->CR |=(1 << 24);
  RCC->CFGR |= (2 << 0);
  SystemCoreClockUpdate();
}
```

Code: `conf.h`

```
#ifndef CONF_H
#define CONF_H
extern void confRCC(void);
extern void confGPIO(void);
extern void setClkPLL(void);
#endif
```

Code: `main.c`

```
#include "conf.h"
void delay_ms(uint32_t delay);

int main (void){
  setClkPLL();
  confRCC();
  confGPIO();
  uint32_t delay_val = 10;
  uint32_t clk = SystemCoreClock;
  while(1){
  GPIOA->ODR ^= (1 << 5);
  delay_ms(delay_val);
}
}

void delay_ms(uint32_t delay){
  for(uint32_t i=0;i<delay;i++){
  for(uint32_t j=0;j<11500;j++);
  }
}
```

4.2 Wiring Diagram

Here's our circuit implementation, you can notice that we use a resistor bar of 220 Ω with a ledbar.

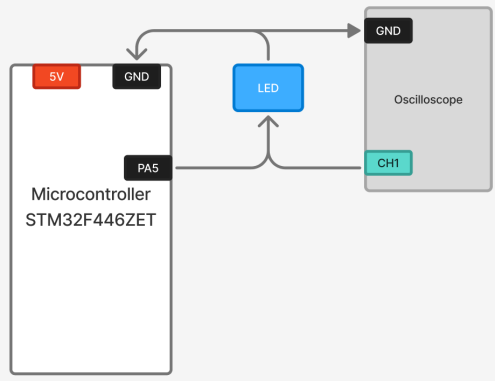

Fig. 2: Vector of Ledbar

4.3 Physical Results

4.3.1 Due to proportions

When we do the practice we obtain that the real max frequency wasn't 180 MHz, it was 90 MHz. To get this value we made a relation 3 doing a delay of 1 ms* due to the different frequencies. The value of 4.6 ms was obtained by a "180 MHz clock" when actually was 90. So we conclude that the maximum clock of peripherals is 90 MHz

*This delay was calculated by our teacher

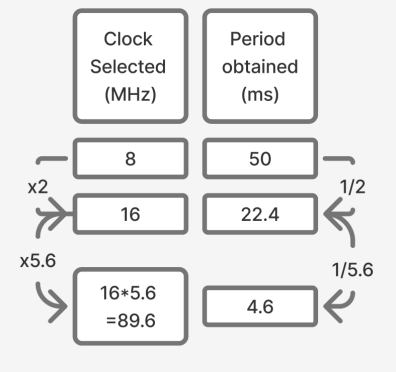

Fig. 3: Relation obtained

4.3.2 Due to program counter

Another way to calculate the frequency is considering the clock cycles/states of the delay of 1 ms 368129 and dividing by the value obtained in the oscilloscope (4.5 ms)

$$\frac{368129 \text{ Clock Cycles}}{1 \text{ Clock Cycles}} = \frac{4.5ms}{X} \rightarrow x = \frac{368129}{4.5ms} = 81.806 \text{ MHz}$$

Considering that is a RISC Architecture, the number of instructions should be approximated the clock frequency. Therefore we get a $\approx 90MHz$. We could test this with the values obtained due to the values obtained.

$$\frac{368129 \text{ Clock Cycles}}{1 \text{ Clock Cycles}} = \frac{22.4ms}{X} \rightarrow x = \frac{368129}{4.5ms} = 16.434 \text{ MHz}$$

$$\frac{368129 \text{ Clock Cycles}}{1 \text{ Clock Cycles}} = \frac{50ms}{X} \rightarrow x = \frac{368129}{4.5ms} = 7.36 \text{ MHz}$$

Q1: How does the LED period change across clock sources?
Debouncing was implemented using a delay of 10 ms.

Q2: Why do we need higher FLASH wait states at higher fCPU?
Because the speed of the embedded flash memory is like 25-30 ns.

Q3: What would break if APB prescalers were left at 1 at 180 MHz?
The entire peripherals because the max of APB1 is 45 MHz while the APB2 is 90 MHz

5 Conclusion

This practice was hard to understand due to frequency, we had already done the configuration so we just need to understand the oscilloscope response, despite of that was so useful to really understand the clock tree and useful for RTOS applications.